Ejercicio Guiado: App de Gestión de Incidencias (*Ticket Tracker*)

Objetivos de la práctica

Vas a construir desde cero una aplicación web para un equipo de soporte técnico. Al finalizar, habrás aplicado:

- **Componentes Standalone** (Padre e Hijo).
- **Comunicación entre componentes** mediante `input()` y `output()`.
- **Signals básicas** (`signal()`, `.update()`).
- **Control Flow nativo de Angular** (`@if`, `@else`, `@for`, `@empty`).
- **Pipes integrados de Angular** (`uppercase`, `titlecase`, `date`).

Instrucciones de desarrollo paso a paso

Paso 1: Crear el modelo de datos (`ticket.model.ts`)

1. Crea un archivo TypeScript llamado `ticket.model.ts`.
2. Define dos tipos de datos personalizados mediante cadenas de texto permitidas:
    - `Prioridad`: solo puede ser `'Baja'`, `'Media'` o `'Alta'`.
    - `Estado`: solo puede ser `'Pendiente'` o `'Resuelto'`.
3. Crea y exporta la interfaz `Ticket` con los siguientes campos:
    - `id`: número identificador único.
    - `titulo`: texto del título de la incidencia.
    - `descripcion`: texto descriptivo del problema.
    - `prioridad`: del tipo `Prioridad` creado anteriormente.
    - `estado`: del tipo `Estado` creado anteriormente.
    - `fechaCreacion`: un objeto de tipo `Date`.

Paso 2: Crear el componente hijo (`TicketCardComponent`)

Este componente representará la tarjeta visual de una única incidencia.

1. **Creación e importaciones:**
    - Crea el archivo `ticket-card.component.ts`.
    - Declara la clase `TicketCardComponent` decorada con `@Component` y configúrala como **Standalone**.
    - Importa `CommonModule` dentro del array `imports` del componente para poder utilizar los Pipes de Angular en su plantilla.
2. **Entradas y Salidas (Inputs & Outputs):**

- Declara una propiedad de entrada llamada `ticket` que sea obligatoria, utilizando la función `input.required<Ticket>()`.
- Declara una salida llamada `cambiarEstado` utilizando la función `output<number>()`. Esta emitirá el ID del ticket.
- Declara una salida llamada `eliminar` utilizando la función `output<number>()`. Esta emitirá el ID del ticket.

3. **Lógica de la clase:**
   - Implementa un método llamado `onCambiarEstado()` que invoque `.emit()` en el output `cambiarEstado`, pasándole el `id` del ticket actual (`this.ticket().id`).
   - Implementa un método llamado `onEliminar()` que invoque `.emit()` en el output `eliminar`, pasándole el `id` del ticket actual.
4. **Plantilla HTML:**
   - Muestra el título del ticket aplicando el pipe `uppercase` (`{{ ticket().titulo | uppercase }}`).
   - Muestra la prioridad aplicando el pipe `titlecase`.
   - Muestra la fecha de creación utilizando el pipe `date` con un formato legible (ejemplo: `'mediumDate'`).
   - Usa la estructura **`@if`** / **`@else`** para evaluar el campo `ticket().estado`:
     - Si el estado es `'Pendiente'`, muestra un mensaje visual de pendiente.
     - Si no, muestra un mensaje de resuelto.
   - Añade un botón que ejecute `onCambiarEstado()` al hacer clic. El texto del botón debe cambiar según el estado actual (si está pendiente debe decir *"Marcar Resuelto"*, si está resuelto debe decir *"Reabrir Ticket"*).
   - Añade un botón para borrar que ejecute `onEliminar()` al hacer clic.

Paso 3: Crear el componente padre (`AppComponent`)

En este componente gestionarás el listado de incidencias y la lógica de actualización del estado.

1. **Configuración del componente:**
   - En `app.component.ts`, añade `TicketCardComponent` dentro del array `imports` del `@Component`.
2. **Estado con Signals:**
   - Crea una propiedad llamada `tickets` que sea una `signal<Ticket[]>()`.
   - Inicializa la Signal con un array que contenga al menos 3 tickets con valores ficticios (variando entre estados `'Pendiente'` y `'Resuelto'`, diferentes prioridades y fechas).
3. **Métodos de actualización de estado:**
   - Implementa el método `toggleEstadoTicket(id: number)`:
     - Utiliza `this.tickets.update(...)` para iterar sobre la lista de tickets.

        - Localiza el ticket con el `id` recibido y conmuta su estado (si era `'Pendiente'` pasa a `'Resuelto'` y viceversa).
        - Recuerda devolver una nueva copia inmutable del objeto modificado (`{ ...ticket, estado: nuevoEstado }`).
    - Implementa el método `eliminarTicket(id: number)`:
        - Utiliza `this.tickets.update(...)` para filtrar la lista y descartar el ticket cuyo `id` coincida con el recibido.

Paso 4: Construir la plantilla principal (`app.component.html`)

1. **Panel superior de estadísticas:**
    - Crea un contenedor informativo que lea la longitud de la lista directamente llamando a la Signal: `{{ tickets().length }}`.
2. **Renderizado del listado con Control Flow:**
    - Utiliza el bloque **`@for`** para recorrer la lista leyendo `tickets()`.
    - Define la propiedad de rastreo obligatoria `track item.id`.
    - Dentro del bucle, inserta la etiqueta `<app-ticket-card>`:
        - Pasa el objeto del ticket actual mediante el property binding: `[ticket]="item"`.
        - Conecta el evento `(cambiarEstado)` para que ejecute el método `toggleEstadoTicket($event)`.
        - Conecta el evento `(eliminar)` para que ejecute el método `eliminarTicket($event)`.
    - Añade el bloque **`@empty`** inmediatamente después del `@for` para mostrar un mensaje informando de que no hay ninguna incidencia en caso de que la lista esté vacía.

Comprobación final

Asegúrate de probar en el navegador lo siguiente:

1. El título se muestra en mayúsculas y las fechas están bien formateadas gracias a los Pipes.
2. Al hacer clic en "Marcar Resuelto", la tarjeta cambia su apariencia y el texto de su botón.
3. Al hacer clic en "Eliminar", la tarjeta desaparece de la pantalla y el contador total se reduce.
4. Si eliminas todas las tarjetas, aparece el mensaje del bloque `@empty`.